



SIMATS
ENGINEERING

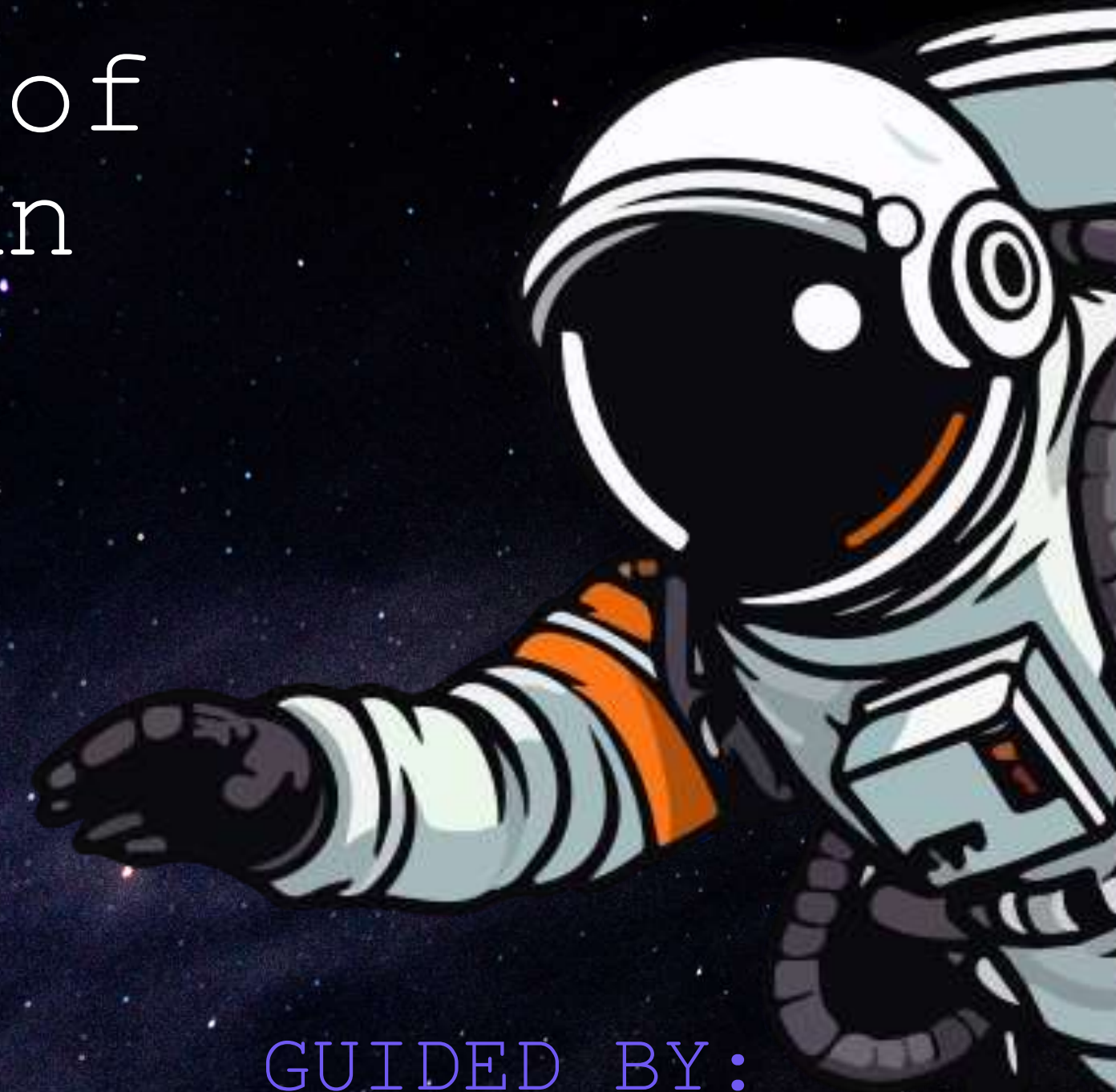


SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Design and Implementation of "Space Mission Rescue" – An Alien Survival Simulator?

C-PROGRAMMING FOR GAME ENGINE
CSA0212

PRESENTED BY:
R.VARSHINI (192525349)
A.MADHURI (192525377)



GUIDED BY:
Dr.V.KARPAGAM



INTRODUCTION



1. Space Mission Rescue is a 2D alien survival and rescue simulation game developed using C programming and SDL3 libraries.
2. The game challenges players to rescue stranded crew members while avoiding and combating hostile aliens.
3. It incorporates key game development concepts such as GUI design, event handling, collision detection, and AI-based enemy movement.
4. The gameplay includes multiple levels, character selection, map selection, health management, shield activation, and shooting mechanics.
5. The project demonstrates the practical application of real-time graphics rendering and interactive software development in a gaming environment.





LITERATURE REVIEW



Authors	Publication Year	Title of the Paper	FINDINGS
Pac-Man Game Development Team	2024	<i>Pac-Man Game Implementation in C</i>	2. Implemented player movement, scoring, and collision detection using C..
Pac-Man Game Development Team	2024	<i>Pac-Man Game Implementation in C</i>	2. Implemented player movement, scoring, and collision detection using C..
Columbia University Research Team	2025	<i>Pac-Man Game Design and Implementation</i>	3. Implemented keyboard input, graphics rendering, and real-time game logic.



PROBLEM STATEMENT



- ✗ Existing 2D games primarily focus on combat and lack mission-oriented rescue objectives.
- ✗ Many games provide limited player interaction and do not include features such as character selection, level progression, and strategic gameplay.
- ✗ Basic survival games often lack intelligent enemy behavior and dynamic difficulty levels.
- ✗ The absence of integrated rescue missions, obstacle navigation, and survival mechanics reduces overall player engagement.
- ✗ There is a need for a lightweight and interactive game that combines rescue operations, alien survival, combat, and progressive challenges within a single platform.





MODULE 1: GUI & SELECTION

MODULE



1. Main Menu Management

- Displays Start and Exit buttons.
- Acts as the entry point of the game.
- Provides navigation to the next game state.

2. Map Selection System

- Allows users to choose from three different maps.
- Enhances visual experience.
- Stores the selected background for gameplay.

3. Player Selection System

- Provides three playable characters.
- Allows user customization.
- Loads the selected player into the game.

4. User Interaction Handling

- Processes mouse click events.
- Detects user selections.
- Controls screen transitions.

5. Interface Rendering

- Loads and displays game assets.
- Renders menus, buttons, maps, and characters.
- Creates an interactive graphical interface.



MODULE 2: PLAYER CONTROL & COMBAT MODULE



1. Player Movement Control

- Controls player movement using keyboard keys.
- Allows smooth navigation during gameplay.
- Restricts movement within game boundaries.

2. Shooting System

- Enables the player to shoot bullets.
- Allows attacks on hostile aliens.
- Improves player survival during missions.

3. Collision Detection

- Detects collisions between bullets and enemies.
- Removes enemies when they are hit.
- Ensures accurate gameplay interactions.

4. Health Management

- Reduces player health when attacked.
- Monitors the player's health status.
- Ends the game when health reaches zero.

5. Combat System

- Combines movement, shooting, and collision features.
- Creates an interactive combat experience.
- Provides smooth and engaging gameplay.



1. Alien Movement

- Aliens move automatically across the game screen.
- Follows predefined movement patterns.
- Creates dynamic and challenging gameplay.

2. Enemy AI

- Aliens chase or attack the player.
- Reacts based on the player's position.
- Makes enemies more intelligent and interactive.

3. Difficulty Progression

- Increases game difficulty at higher levels.
- Speeds up alien movement and attacks.
- Provides a balanced gaming experience.

4. Obstacles & Challenges

- Adds obstacles to increase gameplay difficulty.
- Introduces survival-based challenges.
- Encourages strategic player movement.



5. Gameplay Enhancement

- Combines AI, obstacles, and level progression.
- Creates an engaging and immersive gaming experience.
- Improves overall player interaction and enjoyment.

Output: Smart alien AI with challenging and engaging gameplay.



MODULE 4: MISSION & SCORE MANAGEMENT MODULE

1. Score Tracking

- Tracks the player's score throughout the game.
- Awards points for defeating aliens.
- Displays the current score in real time.

2. Lives Management

- Tracks the player's remaining lives.
- Reduces lives when the player is attacked.
- Ends the game when all lives are lost.

3. Mission Objectives

- Displays mission goals for each level.
- Updates objectives as the player progresses.
- Guides the player to complete challenges.

4. Progress Saving

- Saves the player's game progress.
- Allows the player to continue from the last checkpoint.
- Stores mission completion and score data.

5. Game Result Screen

- Shows the final score after the game ends.
- Displays Win or Game Over message.
- Provides an option to restart or exit the game.

Output: Complete mission management with score tracking, progress saving, and final game results.





TOOLS USED



Module 1 – GUI & Selection Module

- Visual Studio Code
- C Programming
- SDL3
- SDL3_ttf
- BMP Image Assets

Module 2 – Gameplay Engine Module

- Visual Studio Code
- C Programming
- SDL3
- SDL3_ttf
- Collision Detection
- Alien AI Logic
- Game State Management
- Event Handling

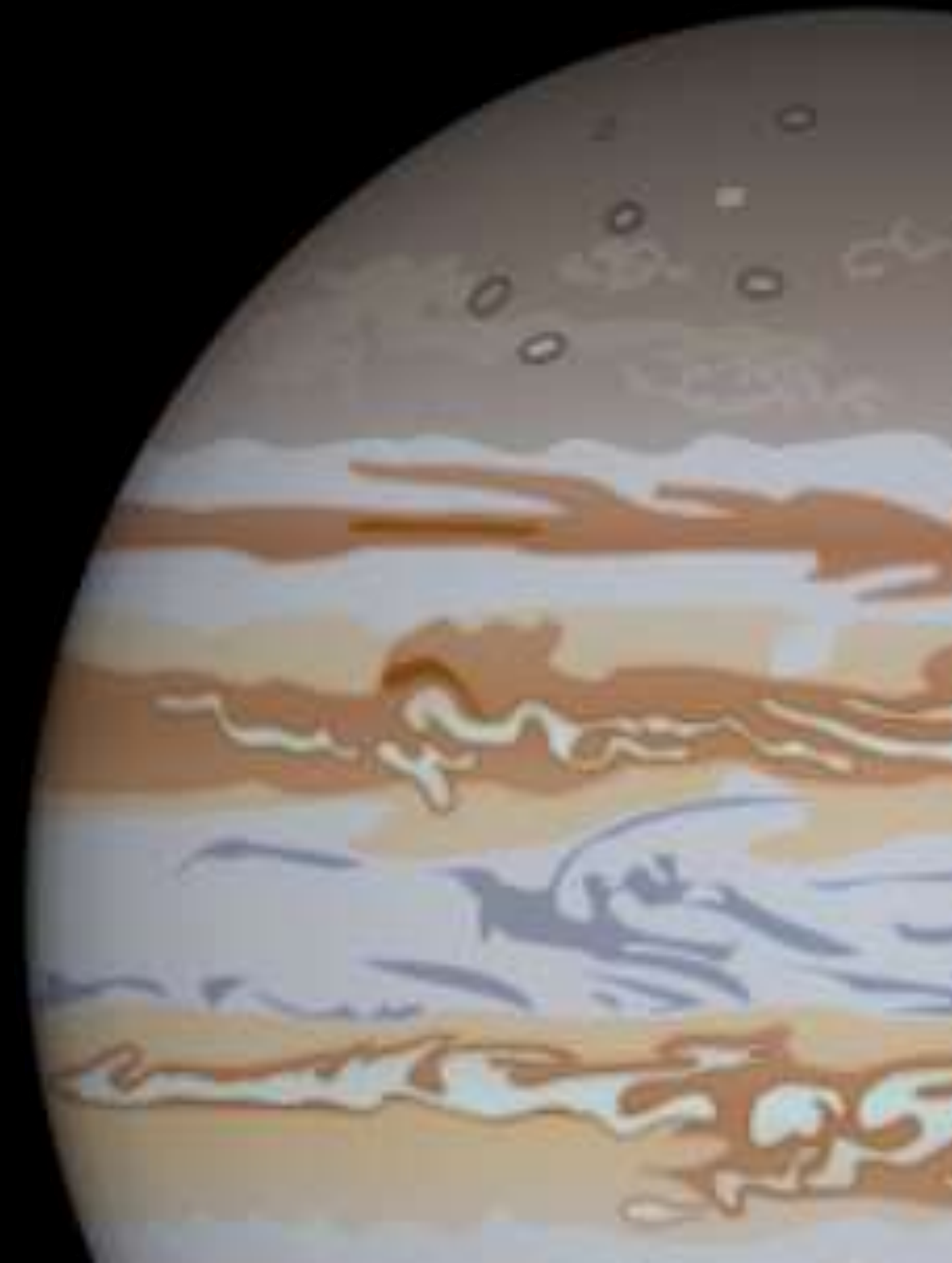




CONCLUSION



1. Successfully developed the Space Mission Rescue game using C and SDL3.
2. Implemented GUI-based map and player selection features.
3. Developed alien AI for dynamic enemy tracking.
4. Integrated shooting, shield, and collision detection mechanisms.
5. Achieved level-based gameplay with mission completion objectives.



FUTURE SCOPE





REFERENCES



- 1.SDL Development Team. (2025). Simple DirectMedia Layer (SDL3) documentation. SDL Foundation. Retrieved from <https://wiki.libsdl.org/SDL3>
- 2.SDL Development Team. (2025). SDL_ttf library documentation. SDL Foundation. Retrieved from https://wiki.libsdl.org/SDL3_ttf
- 3.Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.). Prentice Hall.
- 4.Microsoft Corporation. (2025). Visual Studio Code documentation. Retrieved from <https://code.visualstudio.com/docs>
- 5.Nystrom, R. (2014). Game programming patterns. Genever Benning. Retrieved from <https://gameprogrammingpatterns.com>
- 6.Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.
- 7.Gregory, J. (2018). Game engine architecture (3rd ed.). CRC Press.

Thank
You

